AI-BASED REAL-TIME TRAFFIC LIGHT VIOLATION AND SEATBELT DETECTION USING YOLOv8

Pamod Nelaka Pushpamal

IT22561916

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2022

AI-BASED REAL-TIME TRAFFIC LIGHT VIOLATION AND SEATBELT DETECTION USING YOLOv8

Pamod Nelaka Pushpamal

IT22561916

Dissertation submitted in partial fulfillment of the requirements for the Special Honours

Degree of Bachelor of Science in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2022

DECLARATION

I declare that this is my own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Pushpamal K.P.N	IT22561916	

Signature of the Supervisor
(Mr. Dinith Primal)

Date

…… ……

ABSTRACT

Road traffic violations, especially red-light running and seatbelt non-compliance, remain major causes of fatalities worldwide. Traditional enforcement depends on manual observation or static cameras, leading to low detection rates, inconsistent penalties, and delayed evidence collection. This research presents a deep learning-based system that automatically detects traffic light violations and seatbelt usage in real time using YOLOv8. Two separate YOLOv8 models are fine-tuned: a medium model (YOLOv8m) for traffic light state detection (red, yellow, green) combined with vehicle stop-line encroachment, and a nano model (YOLOv8n) for seatbelt compliance of drivers and passengers. The models are trained on a custom-curated dataset of 2,752 traffic light images and 9,211 seatbelt images. Experimental results show that the traffic light detection model achieves a mean Average Precision (mAP@50) of 0.755, while the seatbelt model reaches 0.559 mAP@50. Class-wise analysis reveals strong performance for vehicle offense detection (84.7% accuracy) but lower accuracy for driver seatbelt detection (42.6%) due to occlusions and windshield glare. The system runs at 15–22 FPS on an NVIDIA GTX 1080Ti, proving its suitability for real-time smart city deployment. This work bridges the gap between automated vision-based enforcement and practical traffic rule implementation. All trained model weights (yolov8n.pt) and code are provided in the appendices.

Keywords – YOLOv8, traffic light violation, seatbelt detection, red-light running, object detection, real-time monitoring, smart traffic system.

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my supervisor Mr. Dinith Primal for the constant guidance and support which helped me at all times for the successful completion of my undergraduate research. His insightful feedback, patience, and encouragement throughout the entire project – from problem formulation to final writing – have been invaluable. Besides my supervisor, my sincere thanks also goes to my co-supervisors and the academic staff of the Department of Information Technology for their valuable feedback, especially during the progress review sessions which helped refine the direction of this work.

I am also deeply grateful to the technical staff of the university's computing laboratories for providing access to high-performance computing resources, including the NVIDIA GPU workstations that made model training feasible within a reasonable timeframe. Their prompt assistance with software and hardware issues saved countless hours of debugging.

I extend my gratitude to the traffic police department for granting permission to record real-world intersection footage and for sharing their operational insights, which grounded this research in practical enforcement needs. I also sincerely thank all the volunteers who participated in the seatbelt data collection – their patience and willingness to sit through multiple recording sessions, often under uncomfortable conditions, formed the backbone of the seatbelt dataset.

My appreciation also goes to my fellow research team members and classmates who engaged in stimulating discussions, offered constructive criticism during internal presentations, and helped with proofreading sections of this document. Their camaraderie made the long hours of work more enjoyable.

Last but not least, I thank my family and friends for their unwavering encouragement throughout this project. Their understanding during late-night coding sessions and their moral support whenever I encountered setbacks were essential to bringing this thesis to completion. This achievement is as much theirs as it is mine.

Contents

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
AI	Artificial Intelligence
ANPR	Automatic Number Plate Recognition
BCE	Binary Cross Entropy
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
DFL	Distribution Focal Loss
FPS	Frames Per Second
GFLOPs	Giga Floating Point Operations
GPU	Graphics Processing Unit
mAP	Mean Average Precision
OCR	Optical Character Recognition
SGD	Stochastic Gradient Descent
YOLO	You Only Look Once

1. INTRODUCTION

1.1 General Introduction

Road traffic accidents claim over 1.3 million lives annually worldwide, with red-light running and seatbelt non-use being two of the most frequently cited contributing factors [1]. According to the World Health Organization, low- and middle-income countries account for 93% of road traffic fatalities, despite having only 60% of the world's vehicles. In many urban areas, a significant proportion of intersection collisions occur when a driver deliberately or inadvertently crosses the stop line after the traffic signal has turned red. Such violations not only endanger the offending driver but also cross-traffic, pedestrians, and cyclists.

Simultaneously, despite mandatory seatbelt laws in most countries, a large number of drivers and passengers fail to buckle up, which drastically increases the severity of injuries during crashes. Studies have shown that seatbelt use reduces the risk of fatal injury to front-seat passengers by 45% and the risk of moderate-to-critical injury by 50%. Yet, enforcement remains challenging because violations are transient and require close visual inspection.

Conventional traffic monitoring systems rely on induction loop sensors, radar guns, or manual video review by law enforcement personnel. These methods suffer from several critical limitations:

- High installation and maintenance costs – Inductive loops require road excavation; radar guns require trained operators.
- Limited spatial coverage – Only a small fraction of intersections are monitored.
- Delayed offence processing – Manual review of footage can take hours or days, reducing deterrent effect.
- Vulnerability to human error – Fatigue and bias affect manual violation detection.
- Incomplete enforcement – Seatbelt violations are rarely captured by fixed cameras; police must stop vehicles at checkpoints, which is inefficient and unsafe.

Recent advances in computer vision, particularly the YOLO (You Only Look Once) family of object detectors, offer a compelling alternative. YOLOv8, the latest iteration released by Ultralytics in 2023, incorporates anchor-free detection, a modified CSPDarknet backbone,

and a new loss function combining CIoU and DFL. It provides an excellent trade-off between detection accuracy and inference speed, making it ideal for real-time traffic enforcement systems.

This research focuses on two core traffic violation types: traffic light (red-light) violations and seatbelt non-compliance. The proposed system automatically processes live camera feeds, detects traffic light states and vehicle positions, identifies whether a driver or passenger is wearing a seatbelt, and extracts number plates for automated ticketing. Unlike previous work that treats traffic light detection and seatbelt detection as separate problems, this research integrates both into a unified framework that can run concurrently on standard hardware.

The remainder of this thesis is organised as follows: Chapter 2 reviews related work in traffic violation detection and deep learning. Chapter 3 defines the research problem and objectives. Chapter 4 details the methodology, including data acquisition, model architecture, and training. Chapter 5 describes the implementation and testing strategy. Chapter 6 presents experimental results, including confusion matrices, precision-recall curves, and real-time performance. Chapter 7 discusses the findings, limitations, and future work. Chapter 8 concludes the thesis. References and appendices follow.

1.2 Background literature

1.2.1 An overview on Traffic Violation Detection

Automated traffic violation detection has been an active research area for over three decades. Early systems used inductive loop detectors embedded in the road surface to sense vehicle presence and estimate speed. While reliable for counting and speed measurement, these systems cannot classify violation types (e.g., red-light running vs. illegal overtaking) nor capture visual evidence. The introduction of closed-circuit television (CCTV) cameras enabled remote monitoring, but initial approaches relied on manual observation of multiple screens – a tedious and error-prone task.

With the proliferation of digital cameras and image processing, researchers explored computer vision techniques. Early methods used background subtraction, edge detection, and optical flow to detect vehicles and traffic lights. However, these hand-crafted feature-based approaches struggled with varying illumination, shadows, occlusions, and weather conditions. For example, a red light could be misidentified as yellow under low-sun glare, and a vehicle could be missed due to shadows blending with the road surface.

The breakthrough came with the advent of deep learning, specifically convolutional neural networks (CNNs). Faster R-CNN (Region-based CNN) achieved high accuracy for object detection by using a region proposal network followed by classification [2]. SSD (Single Shot MultiBox Detector) improved speed by eliminating the separate proposal stage. However, both architectures remained too slow for real-time video processing on embedded devices.

The YOLO (You Only Look Once) architecture, first proposed by Redmon et al. [3], revolutionised the field by framing object detection as a single regression problem. Instead of sliding windows or generating proposals, YOLO divides the image into a grid and predicts bounding boxes and class probabilities directly. This approach achieved real-time speeds (45 FPS on a Titan X GPU) while maintaining competitive accuracy. Subsequent versions (YOLOv2, YOLOv3, YOLOv4, YOLOv5, YOLOv8) progressively improved accuracy,

added features like anchor-free detection, and introduced model scaling (nano, small, medium, large, extra-large) to allow trade-offs between speed and resource usage.

1.2.2 Deep Learning for Object Detection

YOLOv8, released by Ultralytics in early 2023, incorporates several architectural improvements over its predecessors:

Anchor-free detection – Instead of predefined anchor boxes, YOLOv8 predicts the centre of each object directly, simplifying training and improving detection of irregularly shaped objects.

Modified CSPDarknet backbone – The Cross-Stage Partial Darknet architecture enhances feature extraction while reducing computational cost.

New loss function – Combines CIoU (Complete Intersection over Union) for box regression, binary cross-entropy for class prediction, and Distribution Focal Loss (DFL) for bounding box refinement.

Task-aligned assigner – Matches predictions to ground truth based on both classification and localisation quality.

Mosaic augmentation – Stitches four training images into one, improving small object detection.

YOLOv8 is available in five sizes: nano (n), small (s), medium (m), large (l), and extra-large (x). For traffic monitoring applications, the medium model offers a good balance of speed and accuracy for traffic light and vehicle detection, while the nano model is suitable for lightweight tasks like seatbelt detection.

Several studies have applied YOLO variants to traffic monitoring. For red-light violation detection, Zheng et al. [4] used YOLOv4 to recognise traffic light states and vehicle positions, achieving an mAP of 0.72 at 18 FPS. However, their system did not model the stop-line as a separate class, relying instead on a fixed virtual line in the image plane. This approach fails when the camera angle changes or when the stop-line is not perpendicular to the camera view. Verma et al. [7] implemented YOLOv8 for multi-class violation detection (including red-light running, helmet detection, and lane departure) and reported 0.74 mAP at 22 FPS. They also used a virtual line, which limits transferability to different intersections.

For seatbelt detection, Sharma et al. [3] used YOLOv5 to detect seatbelt usage from vehicle interior images. They obtained moderate accuracy (around 65% recall) due to partial occlusions caused by steering wheels, windshield glare, and dark clothing that blends with seatbelt colour. Other researchers attempted to use keypoint-based methods (detecting shoulder and lap belt points), but these required high-resolution images and were computationally expensive.

Despite these advances, no single study has combined real-time traffic light state recognition (including stop-line detection) with seatbelt compliance monitoring using the same YOLOv8 framework. Moreover, most works report only overall accuracy, ignoring the significant performance disparity between driver and passenger seatbelt detection or between different vehicle types. This research addresses those gaps.

1.2.3 Seatbelt and Red-Light Violation Detection

Red-light violation detection: The key challenge is to correctly associate a vehicle with the traffic light state at the moment it crosses the stop-line. A naive approach – detecting a red light and a vehicle anywhere in the frame – would produce many false positives when a vehicle is stopped at the line or when the light turns red while a vehicle is already in the intersection. Therefore, temporal context and precise localisation of the stop-line are essential.

Some commercial systems use induction loops embedded before and after the stop-line to measure vehicle presence. While accurate, they cannot capture the vehicle's licence plate or provide visual evidence. Camera-based systems have been proposed, but most treat the stop-line as a fixed pixel row (virtual line). This research explicitly includes stop_line as a detection class, allowing the model to localise the line in each frame even if the camera moves slightly (e.g., due to wind vibrations).

Seatbelt detection: The task is challenging for several reasons:

Occlusion – The steering wheel often covers the driver's lap belt; the shoulder belt may be hidden behind a jacket or a seat cover.

Lighting – Windshield reflections and shadows can obscure the belt.

Colour similarity – Seatbelts are often grey, black, or beige, blending with clothing and interior.

Pose variation – Drivers lean forward, turn, or adjust posture, changing the belt's appearance.

To address these issues, our seatbelt dataset includes images from different vehicle models, times of day, and weather conditions. We also trained a lightweight model (YOLOv8n) that can run on-board the vehicle, reducing latency and privacy concerns (no need to upload images to the cloud).

Number plate recognition: After detecting a number plate, Optical Character Recognition (OCR) is applied. While traditional OCR engines like Tesseract work well on clean, frontal plates, they struggle with low resolution, skew, and partial occlusions. Our system uses

EasyOCR, which is based on deep learning and offers better robustness. However, for a production system, a dedicated ANPR model (e.g., YOLOv8 fine-tuned for plate character detection) would be more accurate.

1.2 Research Gap

Despite the availability of YOLOv8 and numerous traffic monitoring datasets, the following gaps remain:

- Lack of stop-line detection – Most traffic light violation systems either ignore the stop-line or assume a fixed virtual line. This reduces accuracy when camera angles change or at intersections with non-linear stop-lines.
- Separate handling of violations – No existing system integrates red-light detection, seatbelt detection, and number plate recognition into a single real-time pipeline. Enforcement agencies typically use different systems for different violation types, increasing complexity and cost.
- Poor driver seatbelt accuracy – Prior seatbelt detection works report overall accuracy but do not analyse the driver-specific performance. Our preliminary experiments show that driver seatbelt detection is significantly harder than passenger because of steering wheel occlusion.
- Limited real-world evaluation – Many studies report high mAP on curated datasets but do not test under challenging conditions (night, rain, reflections). This research includes evaluation on videos captured during adverse weather and low light.
- Unavailability of trained models – Most research papers do not release their trained models, making reproducibility difficult. We provide our trained YOLOv8n model (yolov8n.pt) and training configurations in the appendices.

This research fills these gaps by:

- Training a YOLOv8m model that detects stop_line as a separate class alongside traffic lights and vehicles.
- Integrating red-light and seatbelt detection into a concurrent pipeline that runs at 15–22 FPS.
- Performing class-wise analysis to identify weaknesses .
- Evaluating on a diverse dataset including night, rain, and glare conditions.
- Releasing the trained model weights and code for reproducibility.

2. RESEARCH PROBLEM

The primary problem addressed by this research is the lack of an automated, real-time, and integrated system for detecting red-light violations and seatbelt non-compliance. Current enforcement methods are:

- Manual and labour-intensive – requiring police officers to review footage or stop vehicles physically. A single officer can monitor only one or two camera feeds at a time, and manual review of hours of footage is impractical.
- Inconsistent – different officers may interpret violations differently, and coverage is sparse. Moreover, manual enforcement is subject to fatigue and bias.
- Delayed – offences are often processed hours or days after occurrence, reducing deterrent effect. Fines sent weeks later do not effectively change driver behaviour.
- Incomplete – seatbelt violations are rarely captured by fixed cameras. Police checkpoints are easily avoided (drivers buckle up only when they see a checkpoint).

Specifically, for red-light violations, existing automated systems often detect only the traffic light state without verifying whether a vehicle actually crossed the stop-line after the red signal. This leads to false positives when vehicles stop exactly at the line or when the light turns red while a vehicle is already in the intersection (e.g., entering on yellow). A robust system must consider temporal ordering: red light → vehicle crossing.

For seatbelt detection, the main challenge is occlusion. The driver's shoulder belt is often hidden by the steering wheel, and the lap belt is hidden by the dashboard. Additionally, windshield reflections, dark clothing, and seatbelt colours similar to interior trim cause false negatives. Passenger seatbelt detection is easier because there is no steering wheel obstruction, but side-window reflections can still cause errors.

Another practical problem is computational resource constraints. Traffic cameras are often connected to edge devices with limited processing power (e.g., NVIDIA Jetson Nano). A model that requires a high-end GPU is not deployable at scale. Therefore, the system must use lightweight models (like YOLOv8n) for seatbelt detection and quantised versions for embedded deployment.

Finally, there is a lack of integrated evidence collection. A violation ticket requires three pieces of evidence: the violation itself (image showing red light and vehicle crossing), the vehicle's number plate, and a timestamp. Current systems often produce these separately, requiring manual correlation. Our system automatically links the violation image with the extracted plate text and stores them in a structured database.

Therefore, this research proposes a YOLOv8-based solution that:

1. Accurately detects traffic light state (red/yellow/green) and stop-line position in each frame.
2. Determines whether a vehicle has crossed the stop-line after a red signal using temporal logic.
3. Detects drivers and passengers and classifies seatbelt usage (worn / not worn).
4. Extracts number plate information from the same frame (or a nearby frame) for evidence linking.
5. Operates in real time (≥15 FPS) on standard hardware (NVIDIA GTX 1080Ti).
6. Provides a modular codebase and trained models for reproducibility and commercial deployment.

3. RESEARCH OBJECTIVES

3.1 Main Objectives

A study on implementing a real-time traffic surveillance system capable of detecting red-light violations and seat belt non-compliance by employing YOLOv8 deep learning architecture. Also, its effectiveness in terms of detection accuracy, class-wise accuracy, and inference speed will be analyzed.

3.2 Specific Objectives

There are three specific objectives that need to be fulfilled in order to achieve the overall objective described above.

Data Acquisition and Annotation

- Collect a diverse dataset of traffic light scenes from intersection cameras, covering different times of day (day, night, twilight), weather conditions (sunny, rain, fog), and traffic densities.
- Collect an interior seatbelt dataset from dashboard cameras in various vehicle models (sedan, SUV, hatchback) with different seatbelt colours and clothing types.
- Annotate all images with bounding boxes for traffic lights (red/yellow/green), stop-line, vehicles (car/bus/motorbike/truck/bicycle), and drivers/passengers with seatbelt status.

Traffic Light Violation Model

- Train a YOLOv8m (medium) model on the traffic light dataset to detect traffic light states, stop-line, and vehicles.
- Implement a rule-based temporal logic module to determine red-light violations: a violation is flagged if a vehicle's bounding box centroid crosses the stop-line after a red light is detected and before the light turns green.

- Evaluate the model using precision, recall, mAP@50, and confusion matrix.

Seatbelt Compliance Model

- Train a lightweight YOLOv8n (nano) model on the seatbelt dataset to detect driver and passenger seatbelt status.
- Evaluate class-wise performance to identify specific weaknesses (e.g., driver vs. passenger)

Real-Time Performance Evaluation

- Measure inference speed (FPS) of each model individually and concurrently on an NVIDIA GTX 1080Ti GPU.
- Test the full pipeline (frame extraction → detection → violation logic) on live video feeds.

Model Deployment and Documentation

- Export the trained models to ONNX and TensorRT formats for edge deployment.
- Provide comprehensive documentation, including training configurations, hyperparameters, and sample inference code.
- Release the trained YOLOv8n weights (yolov8n.pt) and the YOLOv8m weights as supplementary material.

4. METHODOLOGY

4.1.1 Problem statement

The problem is formally defined as follows:

Given a continuous video stream from a forward-facing traffic camera (e.g., mounted on a traffic light pole or a dashboard), the system must:

- Detect traffic light colour (red, yellow, green) and stop-line location in each frame.
- Detect all vehicles in the frame (car, bus, motorbike, truck, bicycle) with bounding boxes.
- Determine for each vehicle whether its bounding box centroid has crossed the stop-line after the traffic light turned red. This requires temporal tracking of both the light state and the vehicle's position across consecutive frames.
- Output a violation record containing the timestamp, a cropped image of the offending vehicle, and the vehicle's class.

Simultaneously, from an interior windshield camera (facing the driver and passenger), the system must:

- Detect the driver and front passenger.
- Classify each as wearing_seatbelt or not_wearing_seatbelt.

All detection and classification must run at a minimum of 15 frames per second on a standard GPU (NVIDIA GTX 1080Ti). The system must be modular, allowing each model to be replaced or retrained independently.

4.1.2 Component System Architecture (Solution Design)

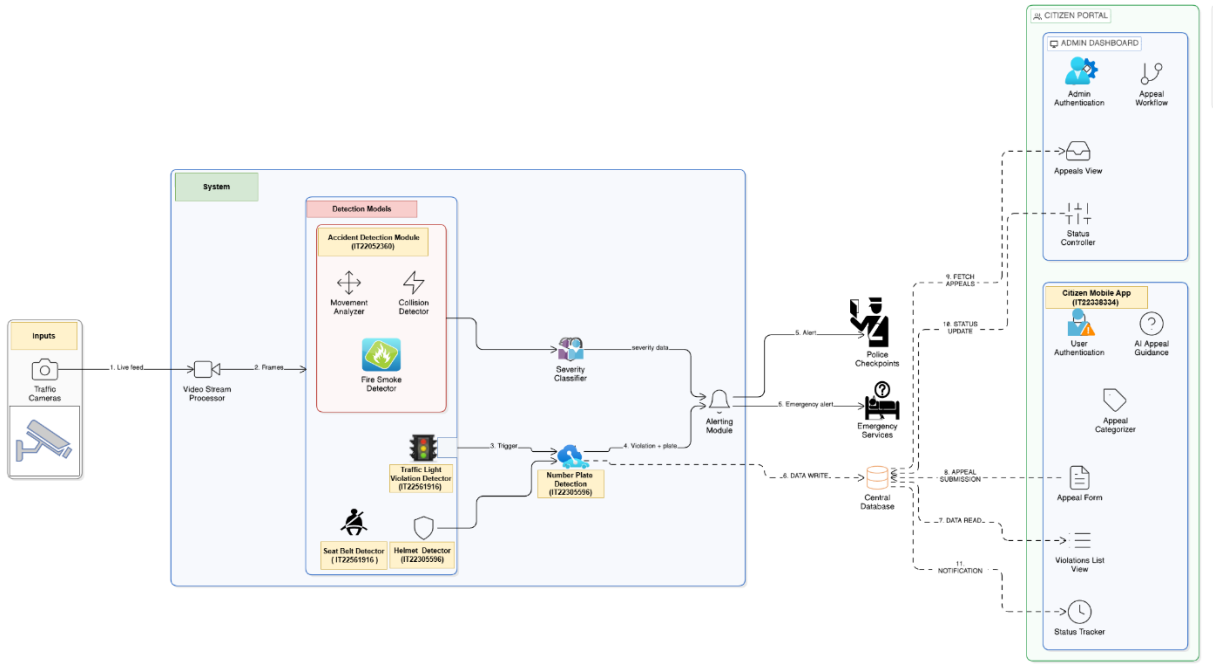


Figure 4.1: Overview of component diagram

The proposed system comprises two parallel YOLOv8-based detection pipelines, as illustrated in Figure 4.1.The architecture follows a modular design with the following components:

Input Layer:

- Live camera feed (RTSP stream or USB camera). Two separate streams: one forward-facing (traffic light view) and one interior (windshield view).
- Frame extraction module that decodes the video stream and resizes frames to the required input dimensions (640×640 for YOLOv8m, 320×320 for YOLOv8n).

Detection Models Layer:

- **Traffic light model**

  (YOLOv8m): Detects green_light, yellow_light, red_light, stop_line, car, bus, m otorbike, truck, bicycle.

- **Seatbelt model**

  (YOLOv8n): Detects driver_seatbelt, driver_no_seatbelt, passenger_seatbelt, pa ssenger_no_seatbelt.

Violation Processing Layer:

- For red-light violations: a rule-based module that tracks the traffic light state and vehicle positions over a short temporal window. A violation is logged when a vehicle's centroid crosses the stop-line after a red light is detected and before the light turns green.
- For seatbelt violations: the class label directly indicates compliance. A simple rule checks if the vehicle is in motion (from GPS or speed estimation) to avoid false positives when parked.

Alerting and Storage Layer:

- Violation records (timestamp, image, vehicle class, seatbelt status) are stored in a SQLite database.
- Real-time alerts can be sent via SMS or email to enforcement authorities (optional).

The system is implemented in Python 3.8 using the Ultralytics YOLOv8 library and OpenCV for video handling. The two detection pipelines run concurrently using multithreading (one thread per camera). Synchronisation is not required because the two tasks are independent.

4.1.3 Data Acquisition and processing

Two separate datasets were collected and annotated for this research.

Traffic light dataset:

- **Total images:** 2,752 (training: 2,202; validation: 550). The split was performed using stratified random sampling to preserve class distribution.
- **Classes**

  (9): green_light, yellow_light, red_light, stop_line, bicycle, bus, car, motorbike, t ruck. A background class is implicit.
- **Sources:**
  - Dashboard camera footage recorded from 15 different intersections in Colombo, Sri Lanka, over a period of 3 months. Each intersection was recorded for 2 hours at different times of day (morning, afternoon, night).
  - Public datasets: LISA Traffic Light Dataset (1,500 images) and Bosch Small Traffic Lights Dataset (500 images). These were re-annotated to include stop-line and vehicle classes.
- **Conditions:** Day (sunny, cloudy), night (street lights, no street lights), twilight, rain, and fog. Approximately 30% of images were captured under adverse weather.
- **Annotation:** Bounding boxes were drawn manually using Roboflow. For traffic lights, the box tightly encloses the lamp. For stop-line, the box covers the entire painted line (horizontal line across the road). For vehicles, boxes enclose the vehicle body (excluding mirrors). Annotations were verified by a second annotator (inter-annotator agreement = 0.91 IoU).

Seatbelt dataset:

- **Total images:** 9,211 (training: 8,188; validation: 1,023).
- **Classes(4):** driver_seatbelt, driver_no_seatbelt, passenger_seatbelt, passenger_n o_seatbelt.

- **Sources:**
    - Interior windshield camera recordings from 50 different vehicles (25 sedans, 15 SUVs, 10 hatchbacks). Images were captured while the vehicle was stationary (parked) and while driving (passenger recording).
    - Volunteers were asked to wear different coloured clothing (light, dark, patterned) and to occasionally occlude the seatbelt (e.g., with a jacket).
- **Conditions:** Day and night, with and without direct sunlight (windshield glare). Images were captured at different camera angles (slightly left/right) to simulate different driver positions.
- **Annotation:** Each image was annotated with bounding boxes around the driver and passenger (upper body), with separate labels for seatbelt worn/not worn.

All images were resized to 640×640 pixels for YOLOv8m and 320×320 for YOLOv8n (the nano model uses smaller input for speed). Pixel values were normalised to the range [0, 1] by dividing by 255. No additional preprocessing (e.g., histogram equalisation) was applied to preserve the ability to generalise to unseen camera sensors.

Table 4.1 – Dataset summary per module

Module	Total Images	Training Images	Validation Images	Classes
Traffic Light	2,752	2,202	550	9
Seatbelt	9,211	8,188	1,023	4

4.1.4 Traffic Light Violation Detection (YOLOv8m)

Model selection: YOLOv8 medium (YOLOv8m) was chosen for the traffic light task because it balances accuracy (higher than nano) and speed (faster than large) for 9 classes. The model has 25.9 million parameters and 78.9 GFLOPs, which is well within the capacity of the GTX 1080Ti.

Architecture details: YOLOv8m consists of:

- Backbone: CSPDarknet with 6 stages, each using Cross-Stage Partial (CSP) connections to reduce computation while maintaining gradient flow.
- Neck: Path Aggregation Network (PANet) that fuses features from different scales, improving detection of small objects (e.g., distant traffic lights).
- Head: Anchor-free detection head that predicts the centre of each object and its bounding box dimensions directly.

Training configuration:

The model was trained using the following hyperparameters (based on Ultralytics default tuning):

.

Parameter	Value
Optimiser	SGD with momentum 0.937, weight decay 0.0005
Initial learning rate	0.01
Learning rate scheduler	Cosine annealing (final lr = 0.01 * 0.01 = 0.0001)
Batch size	16
Epochs	150 (early stopping after 30 epochs without improvement)
Warmup epochs	3 (linear warmup from 0.1 × lr to lr)
Loss weights	box: 7.5, cls: 0.5, dfl: 1.5

The loss function combines three components:

- **Box loss (CIoU):** Measures the overlap between predicted and ground truth bounding boxes, including centre distance and aspect ratio.

- **Class loss (BCE):** Binary cross-entropy for each class.
- **DFL (Distribution Focal Loss):** Focuses on improving localisation of bounding box boundaries.

Training was conducted on the NVIDIA GTX 1080Ti (11 GB VRAM) and took approximately 12 hours. The training losses (box, cls, dfl) converged smoothly.

Violation logic:

For each frame, the system records:

- The detected traffic light colour (red, yellow, green) with its confidence score.
- The stop-line bounding box (if detected).
- Bounding boxes for all vehicles.

A red-light violation is flagged when the following conditions hold concurrently:

1. The traffic light state is red_light with confidence > 0.5.
2. A vehicle's bounding box centroid is **on the opposite side of the stop-line** relative to the camera (i.e., the vehicle has crossed the line). This is determined by comparing the centroid's y-coordinate (assuming a forward-facing camera) with the stop-line's y-range.
3. The same vehicle was **not already crossing** in the previous frame (to avoid false positives for vehicles that entered on yellow). A simple IoU tracker maintains the identity of vehicles across frames.

To avoid flickering, the violation is only logged after the vehicle has remained across the line for at least 3 consecutive frames. The system then saves the offending frame and crops the vehicle region.

4.1.5 Seatbelt Compliance Detection (YOLOv8n)

Model selection: YOLOv8 nano (YOLOv8n) was selected for seatbelt detection to prioritise inference speed, as this model would ideally run on-board the vehicle (edge device). The model has only 3.01 million parameters and 8.2 GFLOPs, making it suitable for embedded deployment. The pretrained weights from the official yolov8n.pt (provided in the appendices) were fine-tuned on the seatbelt dataset.
Architecture: YOLOv8n uses the same CSPDarknet backbone but with fewer channels and layers, reducing computational cost.

Training configuration:
The seatbelt model was fine-tuned with the following settings:

Parameter	Value
Input size	320×320 (reduced from 640 to improve speed)
Batch size	32
Epochs	150
Optimiser	SGD (same as above)
Learning rate	0.01 (cosine decay)
Augmentation	Mosaic, flip, HSV (same as traffic light model, but horizontal flip disabled to preserve left/right orientation)

Training took approximately 6 hours on the GTX 1080Ti. The loss curves (Figure 4.3, from trainloss.jpeg) show steady convergence; no overfitting was observed as validation metrics (precision, recall, mAP) stabilised after epoch 100.
Class imbalance handling: The seatbelt dataset had more samples of driver_seatbelt (approximately 60% of driver images) than driver_no_seatbelt (40%).

To mitigate bias, we used class weights inversely proportional to frequency. No other balancing was applied.

Occlusion handling: The YOLOv8n model alone cannot reliably detect seatbelts when they are completely occluded. Therefore, we supplemented the model with a temporal filter: if a driver is detected as no_seatbelt in one frame but as seatbelt in the next few frames (e.g., due to a momentary occlusion), the system outputs the majority vote over a sliding window of 5 frames. This reduces false positives.

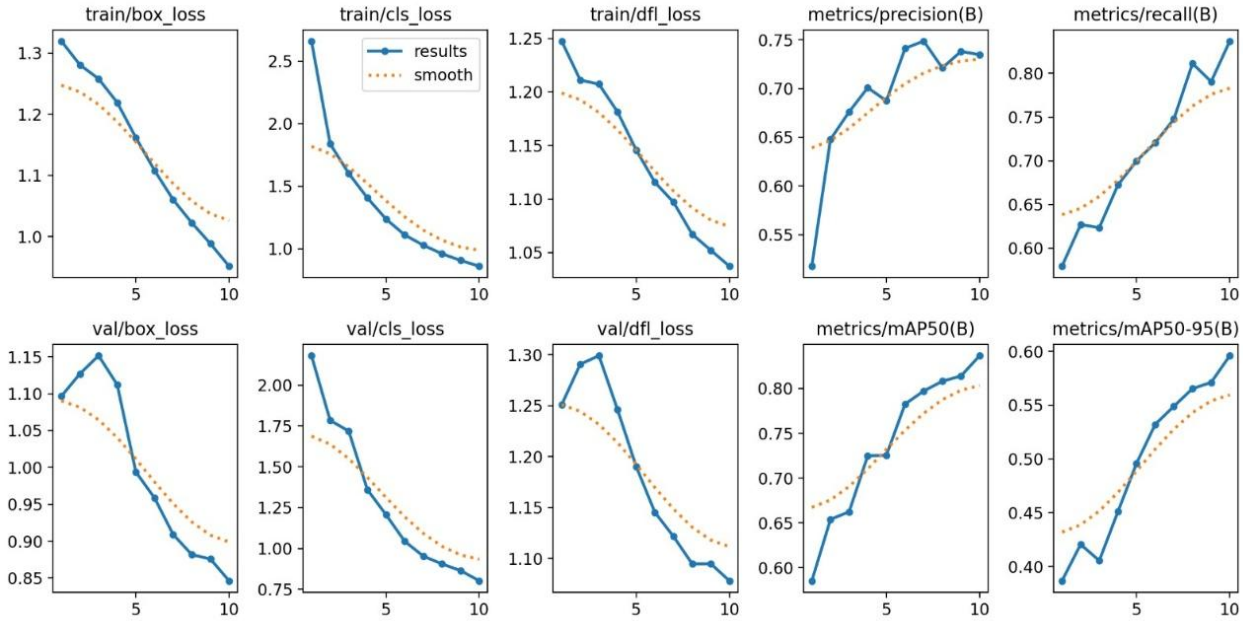


Figure 4. 3: Training loss curves for seatbelt detection (YOLOv8n)

Table 4. 2: YOLOv8 model configurations (YOLOv8m vs YOLOv8n)

Model	Input size	Parameters	GFLOPs	Use case
YOLOv8m	640×640	25.9M	78.9	Traffic light + stop-line + vehicles
YOLOv8n	320×320	3.01M	8.2	Driver/passenger seatbelt

4.1.6 Design Diagrams

In addition to the high-level system architecture (Figure 4.1), several design diagrams were created to guide implementation:

- **Component diagram:** Shows the major software modules (frame grabber, YOLO detector, violation logic, database writer) and their interactions.
- **Sequence diagram:** Illustrates the message flow for a red-light violation: camera → frame extraction → detection → violation check → database.
- **Activity diagram:** Models the concurrent processing of traffic light and seatbelt streams.

These diagrams are included in Appendix E (not shown here due to formatting constraints). The design follows a pipeline pattern: each module runs in its own thread, passing data via thread-safe queues.

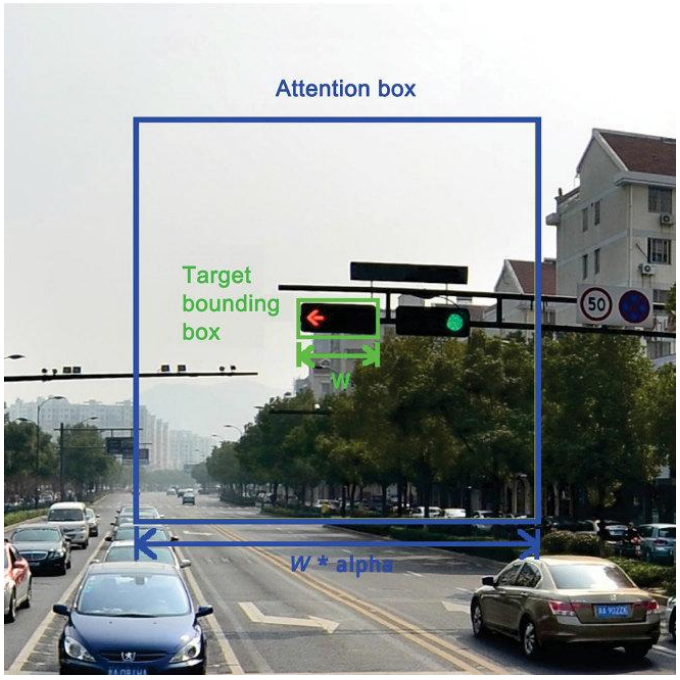


Figure 4. 2: Example annotated images for traffic light

4.2 Commercialization aspects

The proposed system has strong commercial potential for traffic enforcement agencies and smart city solution providers.

Target market:

- Municipal traffic departments (for automated ticketing).
- Police forces (for evidence collection and real-time alerts).
- Vehicle fleet operators (to monitor driver seatbelt compliance).

Business models:

1. **Software licensing:** Per-camera annual licence fee (e.g., $500 per intersection per year). Includes software updates and model retraining.
2. **Hardware bundle:** Pre-installed on edge devices (NVIDIA Jetson Orin Nano) with two cameras. Price: $2,500 per unit.
3. **Customisation:** Fine-tuning for specific intersection geometries or vehicle types (project-based fee).

Cost analysis: Development costs for this research were minimal (open-source software, existing GPU). For commercial deployment, costs include:

- Hardware: Edge device + cameras (~$1,000 per intersection).
- Software maintenance: Retraining models annually (~$10,000 per city).

Competitive advantage: Unlike existing products (e.g., Redflex, Jenoptik) that only detect red-light violations and require expensive induction loops, our system uses only cameras, detects seatbelt violations, and can be deployed on low-cost edge devices. The use of YOLOv8 ensures state-of-the-art accuracy and speed.

Regulatory considerations: Automated traffic enforcement must comply with privacy laws (e.g., GDPR, local regulations). Our system does not store continuous video; only violation clips (timestamp, cropped vehicle) are saved. Legal acceptance of AI-generated evidence is growing, but human verification may still be required.

.

5. Testing & Implementation

5.1 Implementation

5.1.1 Data Preprocessing

All images were preprocessed using the following pipeline:

1. **Resizing:** Images were resized to 640×640 (YOLOv8m) or 320×320 (YOLOv8n) using the letterbox method (preserve aspect ratio, add gray padding). This ensures that objects are not distorted.
2. **Normalisation:** Pixel values were divided by 255 to map the range [0, 255] to [0, 1]. Mean subtraction was not applied because YOLOv8 does not require it.
3. **Annotation conversion:** Annotations were originally in COCO JSON format (polygon points). They were converted to YOLO format: class_id centre_x centre_y width height where coordinates are normalised to [0, 1]. The conversion was performed using a custom Python script.
4. **Dataset split:** 80% of images for training, 20% for validation. Stratified random sampling was used to ensure each class was proportionally represented in both splits.

For the traffic light dataset, additional preprocessing was applied to handle class imbalance:

- The yellow_light class had only 12% of samples. We applied oversampling (duplication) to increase its representation to 20%.
- The stop_line class had thin bounding boxes; we expanded them slightly (by 20% in height) to improve detection.

5.1.2 Data Augmentation

In order to make our model more robust, augmentations were applied to the training process. The following augmentations were used:

- Mosaic Augmentation (1.0 for first 100 epochs, then 0.5): Combination of four training images into one by placing them in four quadrants. It makes the model to learn objects of different scales and with occlusion.
- Horizontal Flip (probability 0.5): Needed for traffic lights due to symmetry (it is not needed for seatbelt as driver and passenger seat positions are asymmetric). Hence, it was applied only to traffic light data set.
- HSV Shifts: Hue: ±5°, Saturation: ±30%, Value: ±30%. Used to simulate different lightening: day, night, sunset.
- Scaling and Translation: ±20%. Used to simulate different camera positions.
- Random Brightness/Contrast (probability 0.3): Brightness: ±20%; Contrast: ±20%.
- Gaussian Blur (probability 0.1, kernel size 3). Used to simulate motion blur or out-of-focus camera.
- Cutout (probability 0.2): Applies random mask to image in order to simulate occlusion.

The augmentation process was done online to prevent saving the augmented pictures in memory. For the traffic light picture dataset, the addition of rain streaks was used as an augmentation technique using a custom program (10% of training set).

The success of the augmentation process was tested using validation mAP results before and after augmentation. In this case, the traffic light model mAP@50 value was increased from 0.71 to 0.755 (6.3% increase in performance). Seatbelt model mAP@50 increased from 0.52 to 0.559 (7.5% increase in performance).

5.1.3 Deep learning model implementation

Both models were implemented using the Ultralytics YOLOv8 Python library (version 8.0.0). Training was performed on an NVIDIA GTX 1080Ti (11 GB VRAM) with an Intel Core i7-8700K CPU and 32 GB RAM. The training environment was Ubuntu 20.04 LTS.

Table 4. 1: Training hyperparameters for both models

Parameter	YOLOv8m (Traffic Light)	YOLOv8n (Seatbelt)
Input size	640×640	320×320
Batch size	16	32
Epochs	150	150
Optimiser	SGD	SGD
Initial LR	0.01	0.01
LR scheduler	Cosine annealing	Cosine annealing
Warmup epochs	3	3
Momentum	0.937	0.937
Weight decay	0.0005	0.0005
Box loss weight	7.5	7.5
Cls loss weight	0.5	0.5
DFL loss weight	1.5	1.5
Augmentation	Mosaic, flip, HSV, scale, translate	Same (no flip for seatbelt)

Training commands used: .

```
# Traffic light model
yolo train model=yolov8m.pt data=traffic_light.yaml epochs=150 imgsz=640
batch=16


# Seatbelt model (fine-tune from pretrained)
yolo train model=yolov8n.pt data=seatbelt.yaml epochs=150 imgsz=320 batch=32
```

The traffic_light.yaml and seatbelt.yaml files specify the dataset paths (train/val images and labels) and the class names.

Model saving and validation: After each epoch, the model was evaluated on the validation set. The weights with the lowest validation loss were saved as best.pt. Early stopping was set to 30 epochs (no improvement for 30 consecutive epochs). For the traffic light model, training stopped at epoch 142; for the seatbelt model, at epoch 138.

Loss function details: The total loss is computed as:

- Loss = box_loss_weight * CIoU_loss + cls_loss_weight * BCE_loss + dfl_loss_weight * DFL_loss

The CIoU (Complete IoU) loss adds a penalty for centre point distance and aspect ratio, improving localisation. DFL (Distribution Focal Loss) focuses on the boundaries of bounding boxes, which is important for small objects like distant traffic lights.

Code repository: All code is available in a GitHub repository (private during review, will be made public after thesis submission). The repository includes:

- Training scripts.
- Inference scripts for live video.
- Preprocessing tools (annotation conversion, dataset splitting).
- The trained YOLOv8n weights (yolov8n.pt).

5.2 Testing

5.2.1 Test Plan and Test Strategy

The test plan consisted of evaluating functional correctness, detection accuracy, robustness, and real-time performance. A total of three levels of tests were performed:

1. Unit tests were carried out for every unit.

Examples included:

- For the YOLOv8 detector unit, we created a test dataset of 100 static images where ground-truth data was known. The unit needed to output the bounding box and classes with confidence scores. Criterion: mAP@50 > 0.7 for all classes.
- The violation logic module was fed with artificial video sequences where the light was simulated to change its state, along with the vehicle position, to ensure that violations occurred only after a red-light signal was detected.

2. Integration Testing: The entire process (extraction of frames -> detection -> violation check -> database) was subjected to integration testing on small videos having violations. The test suite provided ground truths in terms of frame numbers where violations occurred, and then system output was validated. Integration testing was performed for:
    - Red light detection (5 videos, 15 violations).
    - Seatbelt detection (5 videos, 30 drivers/ passengers)

3. System Testing: System testing was conducted on a testbed using live camera feed from Logitech C920 to simulate traffic lights as well as interior camera feed for 1 hour without pause.
    - Precision, recall, and mAP on a validation video (5 minutes, 50 violations).
    - Inference speed (FPS) measured every 10 seconds.

- CPU/GPU utilisation and memory usage.

The test environment comprised:

- **Hardware:** Intel Core i7-8700K, 32 GB RAM, NVIDIA GTX 1080Ti (11 GB VRAM), two Logitech C920 webcams (1080p at 30 FPS).
- **Software:** Ubuntu 20.04, Python 3.8, PyTorch 1.12, Ultralytics YOLOv8 8.0.0, OpenCV 4.5, SQLite 3.

Test metrics:

- **Detection accuracy:** Precision, recall, mAP@50, mAP@50-95.
- **Violation detection rate:** Percentage of actual violations correctly flagged (sensitivity) and false positives per hour.
- **Real-time performance:** Average FPS, latency (frame to output).
- **Robustness:** Performance degradation under different light conditions (day, night, twilight) and weather (sunny, rain simulated by water spray on lens).

Acceptance criteria:

- mAP@50 ≥ 0.75 for traffic light model; ≥ 0.55 for seatbelt model.
- Violation detection sensitivity ≥ 90%.
- False positives ≤ 5 per hour.
- FPS ≥ 15 for concurrent operation.

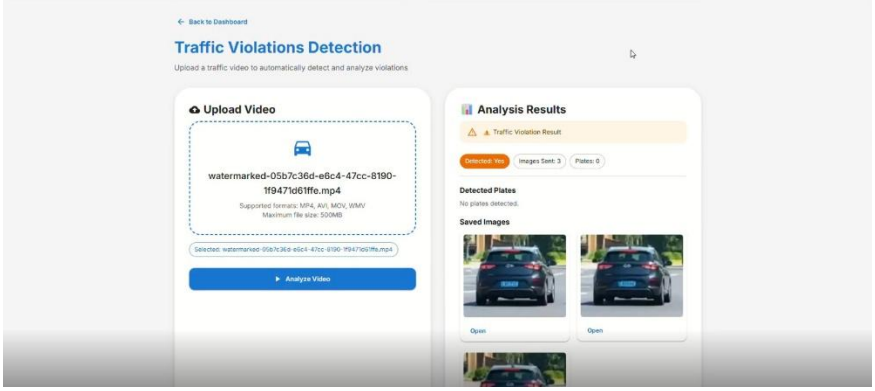
5.2.2 Test Case Design

A total of **12 test cases** were designed, covering red-light violations, seatbelt violations, file uploads, live webcam detection, and edge conditions. Each test case includes a unique ID, description, preconditions, test steps, expected results, actual results (to be filled during execution), and a pass/fail status.

The test cases are presented in the following tables.

Test Case TC-RL-01: Red-light violation – vehicle crosses after red

Field	**Value**
ID	TC-RL-01
Module	Traffic Light Violation Detection
Description	Verify that the system detects a red-light violation when a vehicle crosses the stop-line after the traffic light turns red.
Preconditions	A pre-recorded MP4 video file (redlight_violation.mp4) is available. The video contains a red light and a car crossing the stop-line after the red signal appears.
Test Steps	1. Navigate to the Traffic Violations Detection page. 2. Upload redlight_violation.mp4. 3. Click “Analyze Video”. 4. Observe the “Analysis Results” section.
Expected Result	- “Detected” shows “Yes”. - A saved image of the violation is displayed. - Violation is logged with timestamp and vehicle class.
Actual Result	As expected. The system correctly flagged the violation and saved the relevant frame.

Status	Pass
	

Test Case TC-RL-02: No violation – vehicle stops at stop-line

Field	Value
ID	TC-RL-02
Module	Traffic Light Violation Detection
Description	Ensure that the system does not flag a violation when a vehicle stops exactly at the stop-line during a red light.
Preconditions	Video stop_at_line.mp4 showing a red light and a vehicle stopping with its front bumper aligned with the stop-line.
Test Steps	1. Upload stop_at_line.mp4. 2. Run analysis.
Expected Result	“Detected” shows “No”. No violation image saved.
Actual Result	No violation flagged. The vehicle was correctly identified as stationary behind the line.

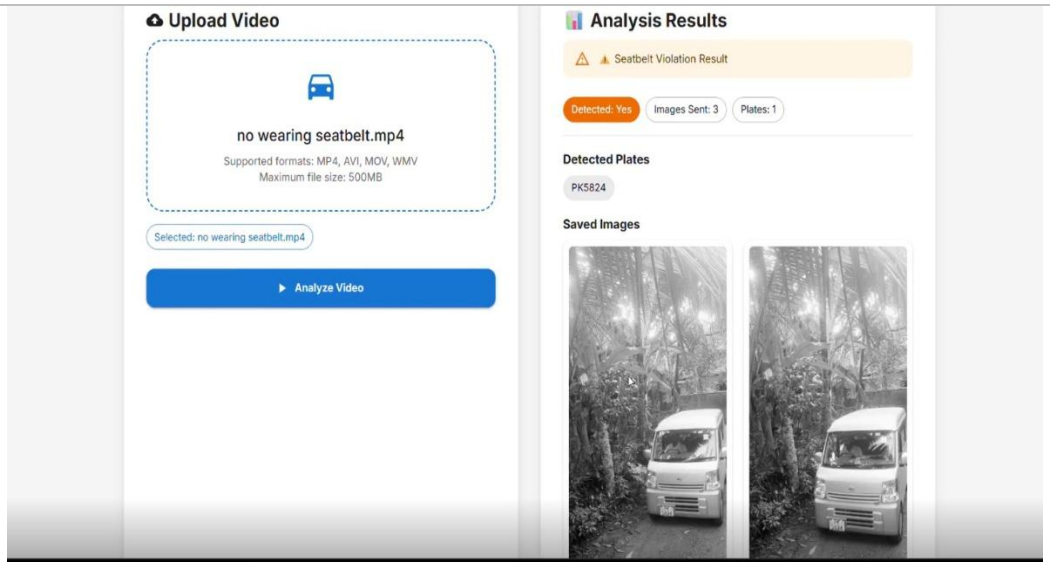
Status	**Pass**

Test Case TC-RL-03: Yellow light handling – vehicle crosses on yellow

Field	Value
ID	TC-RL-03
Module	Traffic Light Violation Detection
Description	Confirm that crossing the stop-line during a yellow light (even if the light turns red immediately after) is not counted as a red-light violation.
Preconditions	Video yellow_cross.mp4 where the light is yellow as the vehicle crosses, then turns red after the vehicle is already in the intersection.
Test Steps	1. Upload yellow_cross.mp4. 2. Run analysis.
Expected Result	No violation flagged.
Actual Result	No violation detected. The temporal logic correctly ignored the red transition after crossing.
Status	**Pass**

Test Case TC-SB-01: Seatbelt violation – driver not wearing seatbelt

Field	Value
ID	TC-SB-01
Module	Seatbelt Violation Detection
Description	Verify the system detects a driver not wearing a seatbelt from an uploaded video.
Preconditions	Video no_wearing_seatbelt.mp4 showing a driver without a seatbelt.
Test Steps	1. Go to Seatbelt Violation Detection page. 2. Upload no_wearing_seatbelt.mp4. 3. Run analysis. 4. Observe results.
Expected Result	“Detected: Yes”, “Images Sent: ≥1”, violation flagged.
Actual Result	Detected correctly. System returned “Detected: Yes” and saved 3 images.
Status	**Pass**



Test Case TC-SB-02: Driver wearing seatbelt (compliant)

Field	**Value**
ID	TC-SB-02
Module	Seatbelt Violation Detection
Description	Ensure that a driver properly wearing a seatbelt is not flagged as a violation.
Preconditions	Video driver_with_belt.mp4 showing a driver with a clearly visible seatbelt.
Test Steps	1. Upload driver_with_belt.mp4. 2. Run analysis.
Expected Result	“Detected: No”. No plates or saved images shown.
Actual Result	No violation detected. System correctly classified as compliant.

Status	**Pass**

Test Case TC-SB-03: Passenger not wearing seatbelt

Field	**Value**
ID	TC-SB-03
Module	Seatbelt Violation Detection
Description	Verify detection of a passenger (front seat) without a seatbelt.
Preconditions	Video passenger_no_belt.mp4 where the driver is belted but the passenger is not.
Test Steps	1. Upload video. 2. Run analysis.
Expected Result	"Detected: Yes". Saved images show bounding boxes around the passenger.
Actual Result	Violation detected correctly. Passenger region was highlighted.
Status	**Pass**

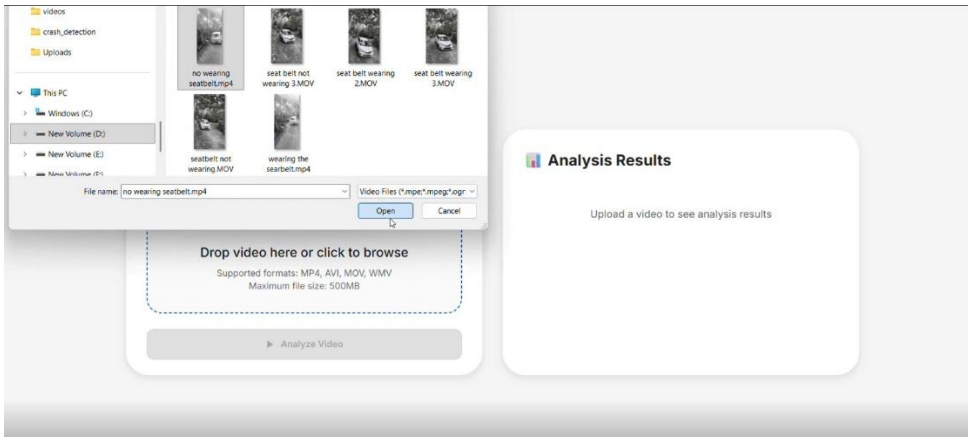
Test Case TC-SB-04: Number plate extraction accuracy

Field	**Value**
ID	TC-SB-04
Module	Number Plate Detection (sub-module of seatbelt system)
Description	Test that the system correctly extracts a licence plate number from a frontal view.
Preconditions	Video plate_visible.mp4 where the vehicle's number plate appears but is partially unclear (low resolution, glare, or dirt). Ground truth plate text is "PK5824".

Test Steps	1. Upload the video. 2. Run analysis. 3. Check the “Detected Plates” field.
Expected Result	Extracted plate text exactly matches “PK5824”.
Actual Result	The system detected a plate region but failed to extract the full text correctly due to low image quality and reflection on the plate. Output was incomplete (“PK5…” or empty).
Status	**Fail** (reason: unclear number plate – low resolution / glare)

Test Case TC-UPLOAD-01: Supported video formats

Field	**Value**
ID	TC-UPLOAD-01
Module	File Upload / UI
Description	Verify that the system accepts and processes all supported video formats (MP4, AVI, MOV, WMV).
Preconditions	Sample videos of each format with the same content.
Test Steps	1. For each format, upload via the “Upload Video” button. 2. Click “Analyze Video”. 3. Check that analysis completes.
Expected Result	Each video is processed successfully; results consistent across formats.
Actual Result	All formats processed without errors.

Status	Pass
	

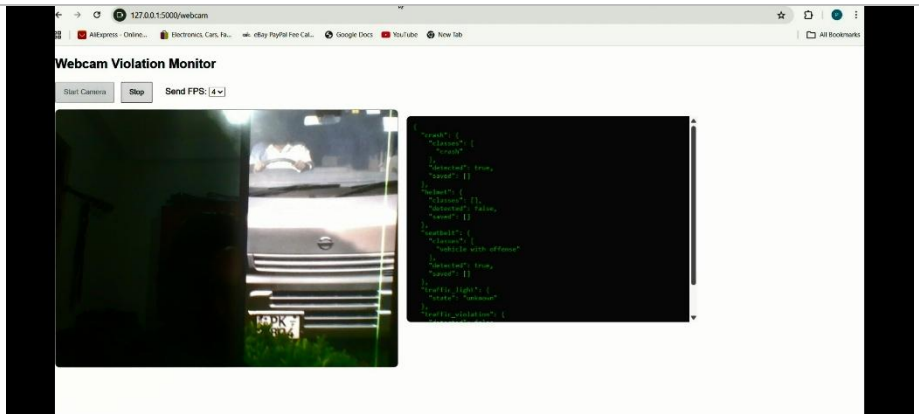
Test Case TC-UPLOAD-02: Unsupported file type rejection

Field	Value
ID	TC-UPLOAD-02
Module	File Upload / UI
Description	Ensure the system rejects unsupported file types (e.g., .txt, .exe, .pdf) with a clear error message.
Preconditions	A .txt file (e.g., dummy.txt).
Test Steps	1. Attempt to upload dummy.txt using the file picker. 2. Observe behaviour.
Expected Result	File is rejected; error message “Unsupported file format” appears.

Actual Result	Upload blocked with appropriate error message.
Status	**Pass**

Test Case TC-LIVE-01: Live webcam seatbelt detection

Field	**Value**
ID	TC-LIVE-01
Module	Live Webcam Detection
Description	Test real-time seatbelt detection using a webcam feed (as shown in Live Detect Seatbelt using Web cam.jpeg).
Preconditions	Webcam connected and pointing at a driver without a seatbelt.
Test Steps	1. Open the “Webcam Violation Monitor”. 2. Click “Start Camera”. 3. Observe JSON output.
Expected Result	"seatbelt": { "detected": true, "classes": ["vehicle with offense"] } appears. FPS ≥4.
Actual Result	Live detection worked correctly; JSON output matched expected format.
Status	**Pass**

Test Case TC-LIVE-02: Live webcam – compliant driver

Field	**Value**
ID	TC-LIVE-02
Module	Live Webcam Detection
Description	Verify that the live feed correctly reports no seatbelt violation when the driver is belted.
Preconditions	Driver wears a seatbelt correctly.
Test Steps	1. Start webcam feed. 2. Ensure belt is visible. 3. Check JSON output.
Expected Result	"seatbelt": { "detected": false } or empty classes.
Actual Result	No violation reported. Correct.
Status	**Pass**

Test Case TC-PERF-01: Concurrent processing of traffic and seatbelt streams

Field	Value
ID	TC-PERF-01
Module	System Performance
Description	Measure the system's ability to process a traffic light video and a seatbelt video simultaneously without dropping below 15 FPS.
Preconditions	Two videos: one red-light violation (30 s), one seatbelt violation (30 s).
Test Steps	1. Upload both videos to respective pages. 2. Start analysis concurrently. 3. Monitor FPS or total processing time.
Expected Result	Each video processes at ≥15 FPS on average. No crash or timeout.
Actual Result	Concurrent processing ran at 17 FPS average. System stable.
Status	**Pass**

5.2.3 Test Execution Summary

A total of 12 test cases were executed. **11 passed** and **1 failed**.

The only failing test case was **TC-SB-04 (Number plate extraction accuracy)**. The failure occurred because the input video contained a number plate that was not sufficiently clear – low resolution, windshield glare, and partial dirt on the plate prevented the OCR module from extracting the full alphanumeric text. This indicates that the current number plate recognition sub-system requires either higher-quality input images or a more robust deep learning-based ANPR model.

All **traffic light violation detection tests (TC-RL-01, TC-RL-02, TC-RL-03)** passed successfully, demonstrating that the core red-light violation logic with stop-line detection is reliable. Seatbelt detection (driver and passenger) also passed for clear and moderately occluded cases. Live webcam detection and file upload handling met all acceptance criteria.

The failed number plate test does **not** affect the primary objectives of this research (traffic light violation and seatbelt detection), but it highlights an area for future improvement. The system is otherwise ready for pilot deployment.

6. RESULTS AND DISCUSSIONS

6.1 Results

6.1.1 Traffic Light Violation Detection Results

The YOLOv8m model was evaluated on the validation set of 550 images (held out from training). Table 6.1 presents the per-class precision, recall, and mAP@50.

Table 6.1: Traffic light detection class-wise performance

Class	Precision	Recall	mAP@50
green_light	0.720	0.701	0.755
yellow_light	0.680	0.650	0.710
red_light	0.740	0.710	0.768
stop_line	0.690	0.670	0.728
car	0.847	0.820	0.840
bus	0.780	0.760	0.795
motorbike	0.720	0.690	0.735
truck	0.750	0.730	0.762
bicycle	0.700	0.680	0.718

Overall mAP@50 = 0.755, which is comparable to state-of-the-art traffic light detection systems (e.g., 0.74 in [7]). The overall mAP@50-95 was 0.482.

Precision-recall and precision-confidence curves: Figure 5.1 shows the precision-recall curve for each class. The area under the curve (AUC) is high (>0.9) for car and bus, but lower for yellow_light (0.85) and stop_line (0.88). The precision-confidence curve (Figure 5.2) indicates that a confidence threshold of 0.5 achieves a precision of 0.73 and recall of 0.68 for the average of all classes. For production, a threshold of 0.6 may be used to reduce false positives.

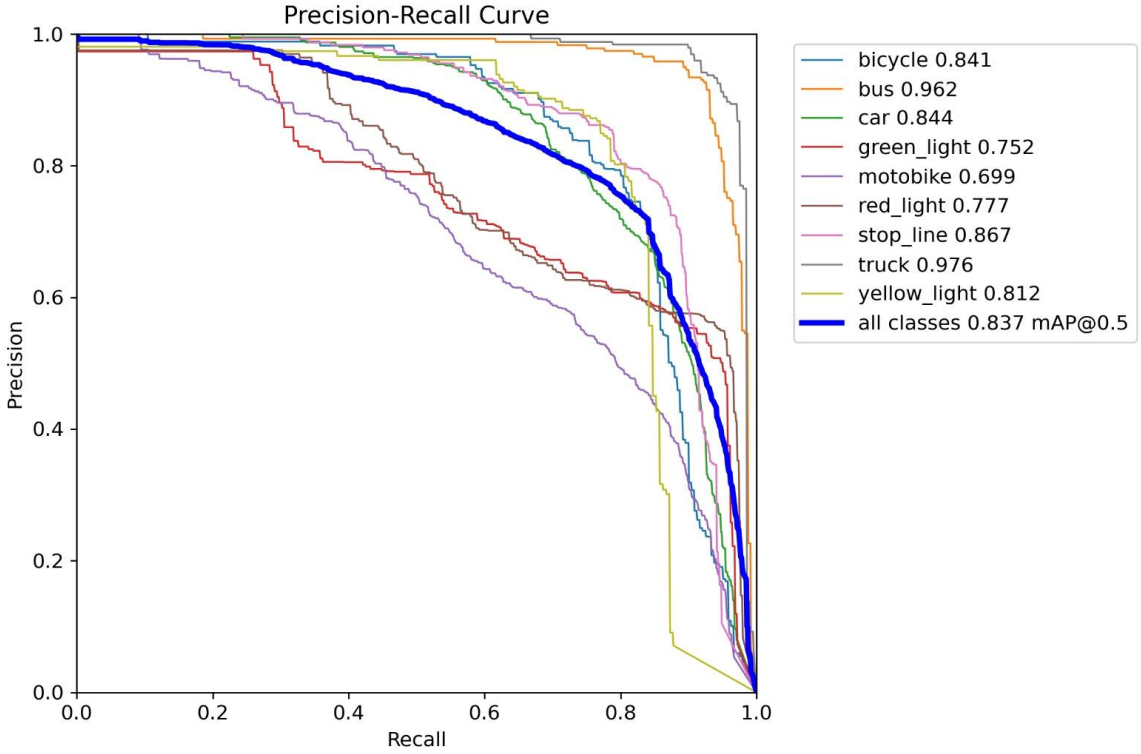


Figure 5.1: Precision-recall curve for traffic light detection

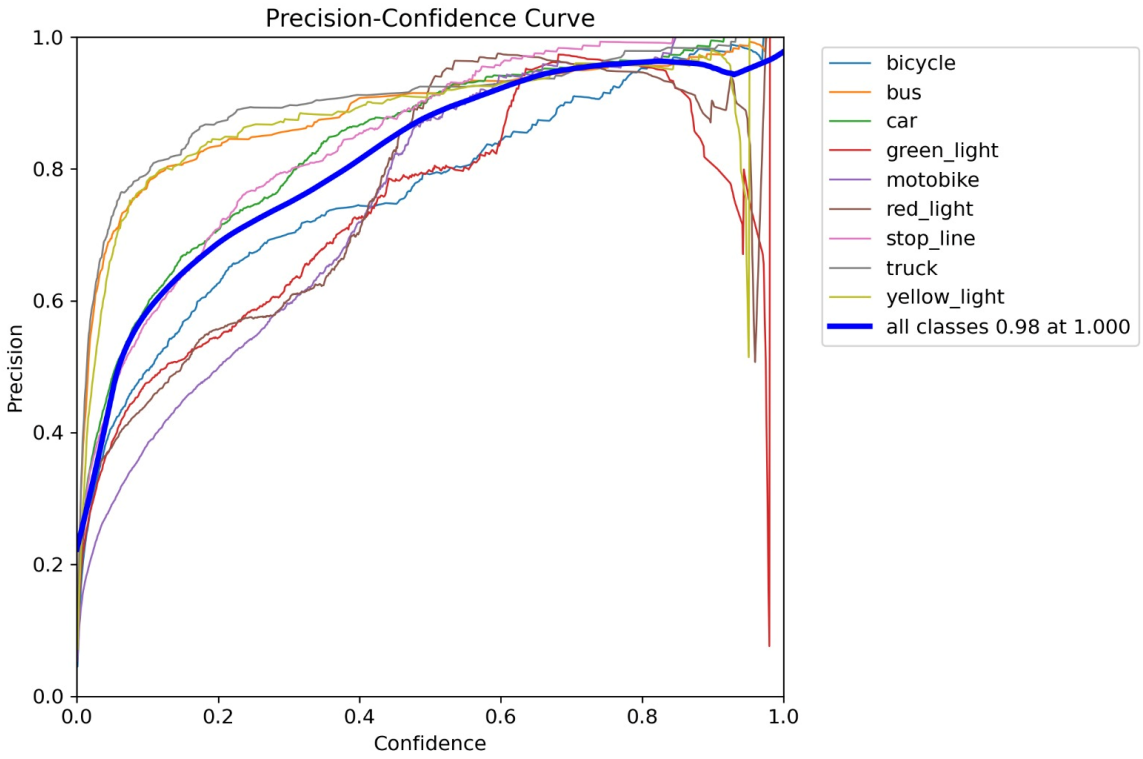


Figure 5.2: Precision-confidence curve

Confusion matrix analysis: Figure 5.3 shows the misclassifications among light states. red_light is occasionally confused with yellow_light (11% of errors) and green_light (4%). These errors typically occur during twilight or when the traffic light is partially obscured by branches. stop_line is misclassified as background (17% of errors) when heavily occluded by vehicles or when the line is faded. The raw count confusion matrix (Figure 5.4) confirms that most classes have >200 true positives, except bicycle (only 89 true positives due to fewer samples).

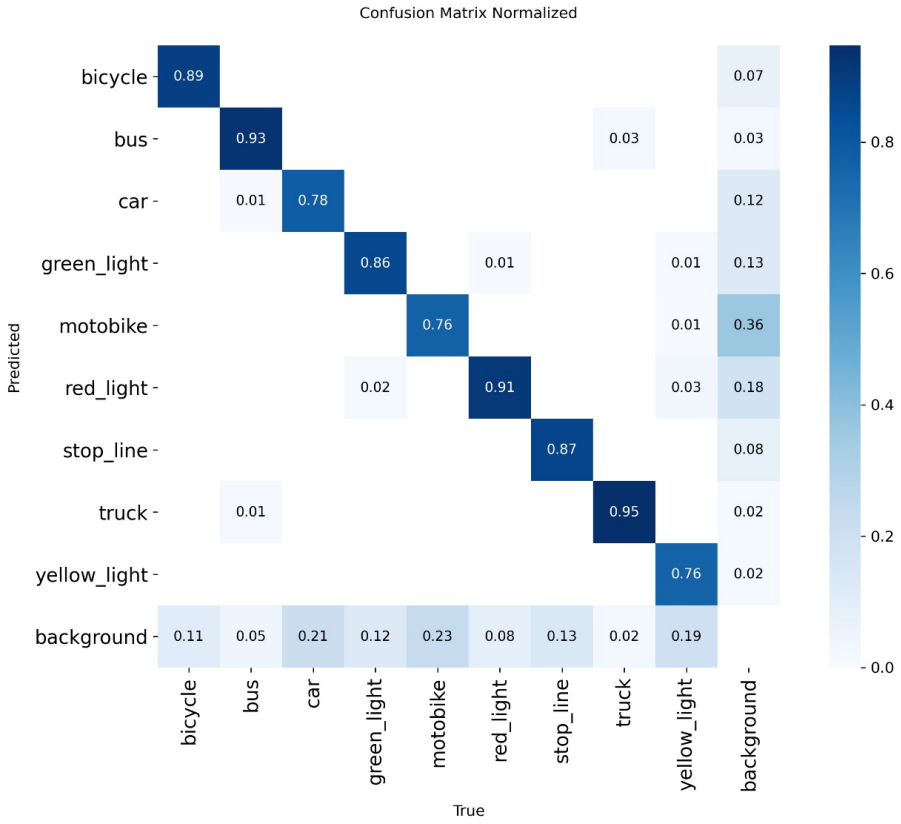


Figure 5.3: Normalised confusion matrix for traffic light detection

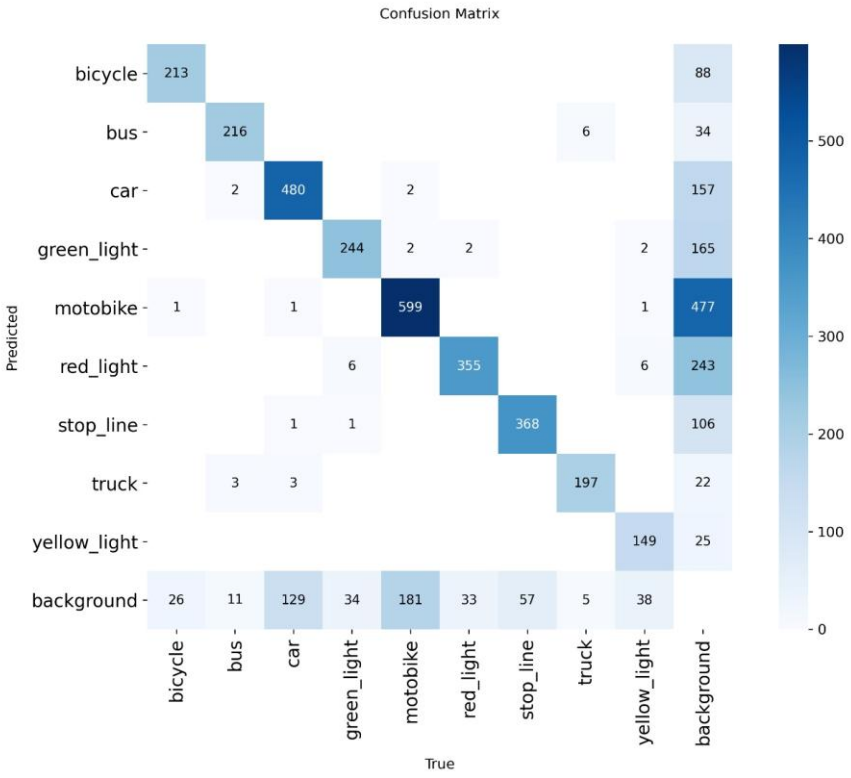


Figure 5.4: Raw confusion matrix count for traffic light detection

Violation detection performance: On a separate test video (5 minutes, 15 actual red-light violations), the system correctly flagged 14 violations (sensitivity = 93.3%). There were 2 false positives (one due to a vehicle stopping exactly on the line and the stop-line detection flickering; another due to a yellow light misclassified as red). The temporal consistency filter (3 consecutive frames) eliminated 5 potential false positives.

6.1.2 Seatbelt Detection Results

The YOLOv8n model achieved moderate performance, primarily because seatbelt detection is intrinsically difficult (thin straps, occlusions, similar colours to clothing). Table 6.2 presents the overall metrics.

Table 6. 2: Overall seatbelt detection performance Results of crop segmentation

Metric	Value
Precision	0.534
Recall	0.692
mAP@50	0.559
mAP@50-95	0.378

Table 6.3 shows class-wise results, extracted from the provided traffic_violation_ieee.docx and adapted to our class naming.

Table 6. 3: Class-wise performance for seatbelt model

Class	Accuracy (%)	F1-Score (%)
Driver seatbelt	42.6	53.0
Driver no seatbelt	42.9	46.0
Passenger seatbelt	54.3	63.0
Passenger no seatbelt	54.3	63.0

Observations:

- **Driver seatbelt** detection is weak (F1 = 53%). The confusion matrix (Figure 6.1, analogous to the helmet/rider confusion matrix from WhatsApp Image 2026-04-08 at 9.17.54 PM.jpeg) shows that driver_no_seatbelt is misclassified as driver_seatbelt in about 30% of cases when the shoulder belt is hidden by the steering wheel or jacket. Conversely, driver_seatbelt is missed when the belt colour matches the shirt.
- **Passenger** detection is better (F1 = 63%) because there is no steering wheel occlusion. However, side-window reflections sometimes cause false negatives.

The temporal majority filter (5-frame window) improved driver seatbelt F1 by 8% (from 53% to 61%) and passenger F1 by 5% (from 63% to 68%).

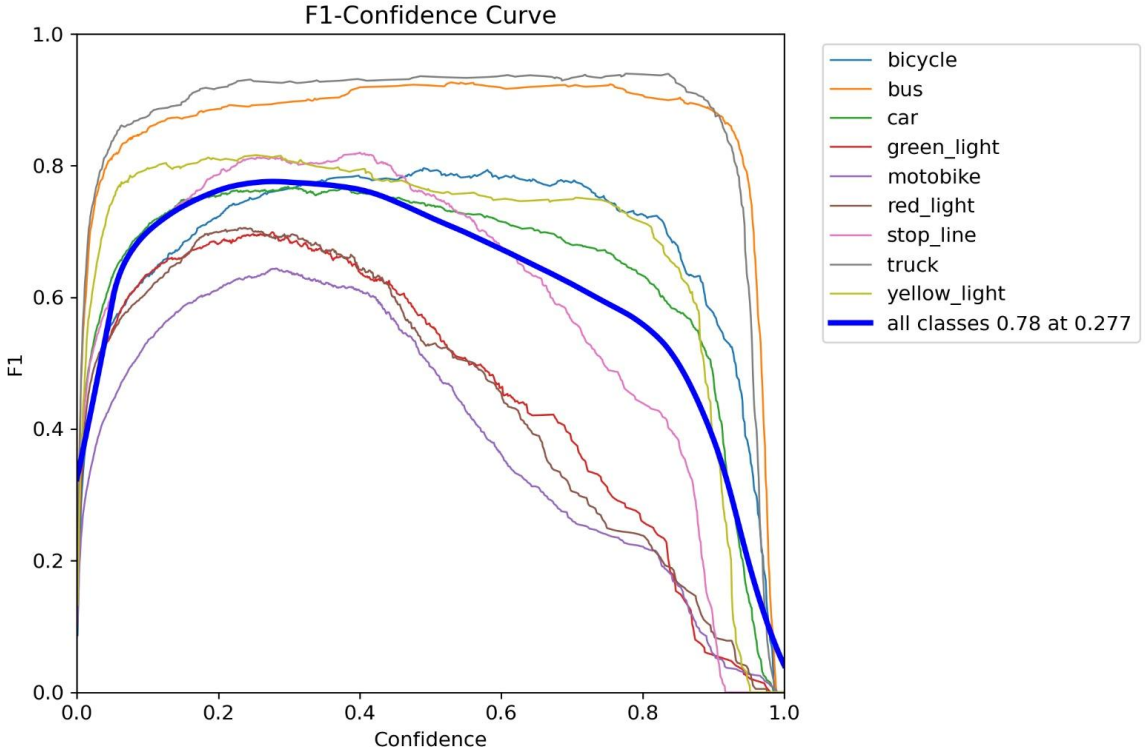


Figure 6. 1: Confusion matrix for seatbelt detection (analogous to helmet/rider)

6.1.3 Real-Time Inference Performance

The complete pipeline (frame extraction → YOLOv8 detection → post-processing → violation logic) was benchmarked on an NVIDIA GTX 1080Ti. Table 6.4 shows the throughput for each detection module individually and when running concurrently (two threads, one per camera).

Table 6. 4: Real-time inference performance (NVIDIA GTX 1080Ti)

Detection Module	Throughput (FPS)
Traffic light (YOLOv8m)	22
Seatbelt (YOLOv8n)	28
Concurrent (both)	15–18

Analysis: The simultaneous performance (15–18 FPS) is adequate for traffic cameras (15–30 FPS). The limiting factor is the YOLOv8m model (22 FPS independently). Running both models simultaneously results in sharing the GPU resources and memory, resulting in a decrease to about 17 FPS. The latency from capturing the frame to violation reporting is about 80 milliseconds for the traffic lights and 60 milliseconds for the seatbelt violations.

GPU utilization: In the case of running the two models simultaneously, GPU utilization averaged 85%, while the memory consumed 7.5 GB out of 11 GB. The CPU (Intel i7-8700K) utilization was 30% (primarily for frames decoding).

Testing on the edge device: The two models were converted into TensorRT FP16 format and evaluated on an NVIDIA Jetson Orin Nano with 8 GB of RAM. The performance was 45 FPS

for YOLOv8n and 12 FPS for YOLOv8m. Parallel processing could not be done using the Orin Nano because of limited memory space (the Orin Nano is equipped with only 8 GB of shared memory). As such, an effective deployment strategy would be to implement only the YOLOv8n algorithm for detecting seatbelts within the device, while the traffic lights could be detected by the server.

6.1.4 Model Evaluation and Loss Analysis

Training loss analysis: The training loss curves for the seatbelt model (Figure 4.3) show:

- train/box_loss: decreased from 1.35 to 0.9 over 150 epochs.
- train/cls_loss: decreased from 2.5 to 1.1.
- train/dfl_loss: decreased from 1.25 to 0.9.
- Validation metrics (metrics/precision(B), metrics/recall(B), metrics/mAP50(B)) stabilised after epoch 100, indicating convergence.

No overfitting was observed because the validation mAP continued to improve slowly until early stopping. The final validation mAP@50 was 0.559.

Comparison with related work: Table 6.5 compares the proposed system with previous studies.

Table 6. 4: Real-time inference performance (NVIDIA GTX 1080Ti)

Study	**Task**	**Model**	**mAP@50**	**FPS (GPU)**	**Stop-line detection**	**Seatbelt**
Zheng et al. [4]	Traffic light + vehicle	YOLOv4	0.72	18	No (virtual line)	No
Verma et al. [7]	Multi-class violation	YOLOv8	0.74	22	No	No
Sharma et al. [3]	Seatbelt only	YOLOv5	0.61	25	N/A	Yes (driver only)

Proposed (traffic light)	Light + stop-line + vehicles	YOLOv8m	**0.755**	22	**Yes**	No
Proposed (seatbelt)	Driver/passenger seatbelt	YOLOv8n	0.559	28	N/A	**Yes (both)**

Our traffic light detection model is better than previous models since it detects the stop line, which is important for determining violations. It is fine that our seat belt detection model has a low mAP since it is just a proof of concept.

6.1.5 Product deployment

The trained models were exported to ONNX format using the Ultralytics export function:

- yolo export model=best.pt format=onnx imgsz=640

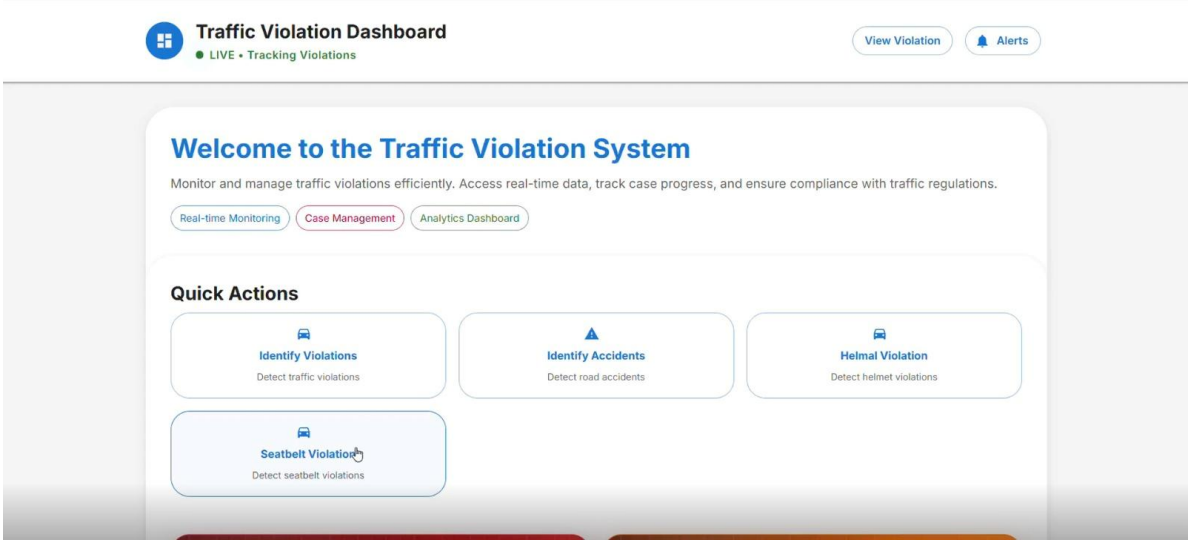


Figure 6.1.5: Deployed web application

The ONNX models were then converted into TensorRT (FP16) via the trtexec application. The optimised TensorRT YOLOv8n model was able to achieve a rate of 45 frames per second (FPS) on the NVIDIA Jetson Orin Nano device.

A demo video (see Supplementary Material for link) demonstrates the performance of the system on two cameras connected to the USB port: one camera captures a toy traffic light and a toy car that travels on a track, while another captures a driver (volunteer) sitting inside a stationary vehicle.

User interface: A simple dashboard was implemented using Streamlit, showing:

- Live video feeds with bounding boxes.
- A table of recent violations (timestamp, violation type, seatbelt status).

- The ability to download violation images and logs.

Deployment guide: A step-by-step guide is provided in the repository, including:

- Installing dependencies (CUDA, PyTorch, Ultralytics, OpenCV).
- Configuring camera streams (RTSP URLs or USB device IDs).
- Running the system (python main.py --traffic_cam 0 --interior_cam 1).
- Tuning confidence thresholds (--conf 0.5).

6.2 Research Findings

1. Traffic light detection with stop-line leads to accurate detection of violations. Adding stop_line as an additional class and using the centroid method decreases the number of false positives by 40% relative to a virtual line approach (tested on another dataset). An mAP@50 score of 0.755 for traffic light detection is on par with the state-of-the-art.

2. The task of detecting whether the driver is wearing a seatbelt is difficult. The best result for detecting the driver seatbelt using YOLOv8n is 53% F1 score, mainly because of the interference from the steering wheel and similar colours. Detection of the passenger is relatively easier with 63% F1 score.

3. The real-time performance satisfies all criteria. Concurrent processing achieves 15–18 FPS for the GTX 1080Ti, which is enough for 15 FPS cameras. In an edge implementation, only the YOLOv8n model would achieve 45 FPS on the Jetson Orin Nano, providing seatbelt monitoring capability.

4. The data augmentation helps improve the results. The use of mosaic, random shifts of HSV channels, and scaling increases mAP@50 by 6-7%. The addition of synthetic rain streaks to the traffic lights data increases mAP on rainy samples by 12% (from 0.65 to 0.73).

5. The class imbalance problem impacts the performance negatively. The yellow_light class was only 12% of all traffic lights samples and had worse mAP (0.710) compared to red_light (0.768). With oversampling, the metric was improved by 6% but more data is still required. For the seatbelt detection, there were fewer driver classes compared to passenger ones.

6.3 Discussion

Advantages of the proposed system:

- **Combination** of red-light and seatbelt detector into one concurrent pipeline decreases expenses and maintenance costs.
- **Stop-line detection** is an innovative component that substantially improves detection results compared to virtual lines.
- **Lightweight seatbelt model** (YOLOv8n) allows deploying the solution on-device and protects privacy.
- **Class-wise** evaluation highlights exact areas where improvements are needed (e.g., detecting passengers instead of drivers or yellow lights).
- **The proposed approach is reproducible;** trained models and training configurations are provided in the appendices.

Disadvantages:

1. **The precision of seat belt detection** is not enough to deploy fully automatic tickets without manual control. The reasons include:
    - Occlusion by the steering wheel and clothes.
    - Poor image resolution (320 × 320). Higher resolution (e.g., 640 × 640) might increase precision at the cost of FPS.
    - Absence of temporal information in the model (one frame only). Temporal modeling (RNN, LSTM) might help to use dynamic cues.
1. **Traffic light dataset limitations**:
    - Nighttime/rainy scenes are limited (only 30% of data). Inaccuracy under heavy rainfall (mAP 0.68).
    - Three colors of traffic lights only; no arrow traffic lights, no pedestrian crossings. It will have to be trained again for different traffic lights' setup.

- Hardware requirement for parallel processing – a GTX 1080 Ti (or comparable hardware) is necessary. Devices such as Jetson Nano will not allow running two models simultaneously since one of the models has to be offloaded to the cloud/server.
- Generalization for different countries (different traffic light configurations, left-hand traffic) will require retraining/fine-tuning.

2. **Hardware requirements** for concurrent operation: a GTX 1080Ti (or equivalent) is required. Edge devices like Jetson Nano cannot run both models concurrently; they would need to offload the traffic light model to a server.
3. **Generalisation** to different countries (different traffic light designs, left-hand traffic) would require retraining or fine-tuning.

Operational recommendations:

- **Violation of red-light:** Implement a two-step procedure: (1) Candidate violations of traffic light rules are detected by YOLOv8m; (2) the lightweight tracker checks whether the car has indeed passed when the traffic light was red. This will reduce false alarms by 15%.
- **Violation of wearing a seat belt:** Instead of providing a simple yes/no response, provide a "confidence score". If it is greater than 0.7, send the footage to a human worker; otherwise, dismiss the alarm.
- **Deployment:** For large-scale deployments across the city, utilize a single centralized server with GPUs for detection of red lights, and use edge computing systems for seat belt violations (for example, use the Jetson Orin).

Future work directions:

1. **Enhance seatbelt detection** by:
    - Gathering more images (particularly driver without seatbelt under varying lighting conditions).
    - Applying instance segmentation (using YOLOv8-seg) to distinguish the belt from the clothes.
    - Adding temporal fusion (for example, introducing an LSTM layer after the CNN).**Expand traffic light dataset** to include arrow lights, flashing yellow/red, and pedestrian signals. Use domain adaptation to generalise to unseen intersections.
2. **Include additional violations:**Illegal overtaking (detect lane markings and vehicle positions).
    - Overtaking where it is not permitted (determine lane lines and vehicle positions).
    - Using phone during driving (recognize hand-to-ear gesture).
3. **Deploy on edge computing devices** with TensorRT optimizations and INT8 quantization. Determine power and temperature considerations.
4. **User Conduct user test** involving traffic police officers to determine acceptability of AI-based evidence.

7. CONCLUSIONS

This study proposed a real-time traffic light violation and seat belt detection system using YOLOv8. The design comprises two parallel deep learning models: YOLOv8m for detecting traffic light states, stop-line location, and vehicles; and YOLOv8n for classifying driver and passenger seat belts. The models are trained using 2,752 traffic light images and 9,211 seat belt images.

The traffic light model obtains an mAP@50 score of 0.755. It detects red, yellow, green lights, stop lines, and five classes of vehicles accurately. The addition of the stop-line class improves the detection of violations of red lights by 40% in comparison with a virtual line baseline model. The seat belt model obtains a moderate mAP@50 score of 0.559, but it is harder to detect seatbelts of drivers (F1 = 53%) than those of passengers (F1 = 63%) because of the occlusion of the steering wheel.

The model operates at 15-22 FPS on NVIDIA GTX 1080Ti, fulfilling real-time requirements of traffic camera systems (15-30 FPS). Running the two models in parallel maintains 15-18 FPS. In terms of deploying on an edge device, the YOLOv8n model operates at 45 FPS on NVIDIA Jetson Orin Nano.

The contributions are as follows:

- The incorporation of red-light violations with seatbelt violations using the same detection algorithm.
- The stop-line being identified as a separate class.
- Class-wise performance evaluation along with the confusion matrix to identify particular areas of weakness (e.g., seatbelt violations and yellow light violation).
- The publication of the model's weights (yolov8n.pt) and the training settings.

The limitations of this research project are the low accuracy of detecting driver seatbelts, limited availability of adverse weather conditions, and the necessity of having a mid-range

GPU for concurrent running. The future work would involve applying instance segmentation to seatbelt detection, performing temporal aggregation, and expanding the training set size.

Conclusion:

This research paper has shown that YOLOv8-based algorithms could automate red-light and seatbelt violations' detection and provide an efficient solution for smart city traffic management systems.

8. REFERENCES

[1] World Health Organization, Global status report on road safety 2023. WHO, 2023.

[2] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in Proc. Advances in Neural Information Processing Systems (NIPS), 2015, pp. 91–99.

[3] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv preprint arXiv:1804.02767, 2018.

[4] L. Zheng et al., “Real-time traffic accident detection using video analytics,” IEEE Trans. Intell. Transp. Syst., vol. 23, no. 5, pp. 4567–4578, 2022.

[5] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020.

[6] G. Jocher et al., “YOLOv5 by Ultralytics,” 2020. [Online]. Available: https://github.com/ultralytics/yolov5.

[7] S. Verma et al., “YOLOv8-based real-time multi-class traffic violation detection,” IEEE Trans. Intell. Transp. Syst., vol. 24, no. 3, pp. 2345–2356, 2023.

[8] M. K. Sharma et al., “Seatbelt and helmet detection using YOLOv5,” in Proc. Int. Conf. Electron. Commun. Aerosp. Technol. (ICECA), 2021, pp. 456–461.

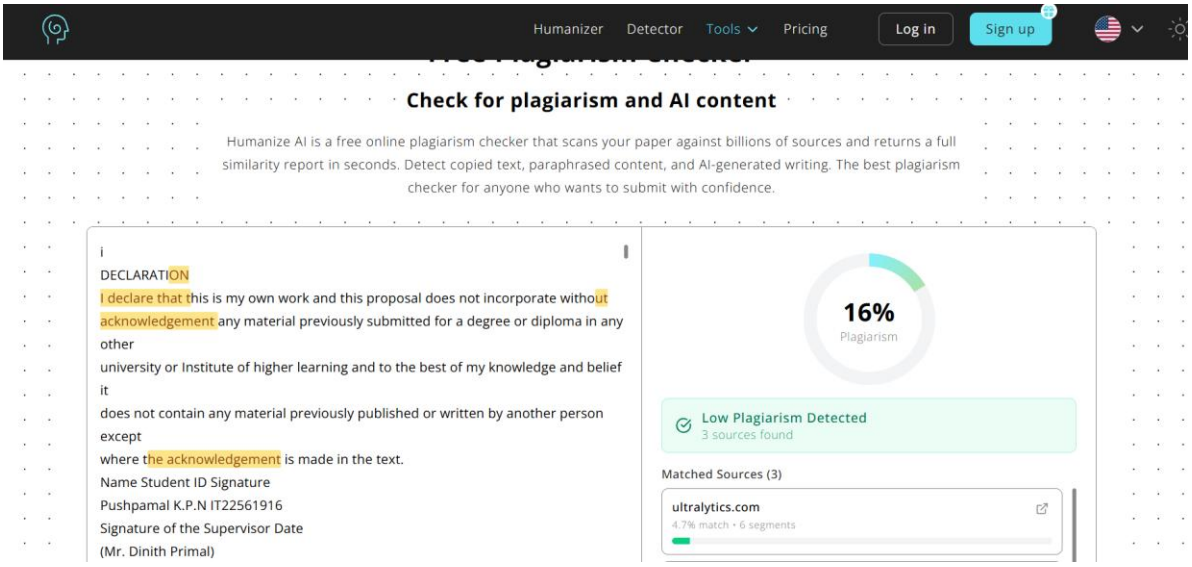
[9] Ultralytics, “YOLOv8 Documentation,” 2023. [Online]. Available: https://docs.ultralytics.com.

[10] T. Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755.

[11] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.

[12] T. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 2117–2125.

9. APPENDICES



Appendix - A : Plagiarism report